

Member 3 — Similarity Features + Machine Learning

Business Entity Resolution Hackathon: Input, Output & Modelling Documentation

1. Member 3 Role

The supplied project guide assigns Member 3 the role of ML/NLP Engineer. The job is to calculate similarity features for candidate business pairs and use a classifier to estimate whether each candidate pair is a true match. The expected output is a match probability or classification for each candidate pair.

2. Position in the Full Pipeline

Raw data → Preprocessing (Member 1) → Candidate generation (Member 2) → Similarity + ML (Member 3) → Evaluation & integration (Member 4) → Final submission.

Member 2 creates plausible candidate pairs. Member 3 should score those candidates. Member 4 handles final threshold tuning, integration, evaluation and submission outputs.

3. Inputs to Member 3

Input	Purpose	Fields used in practice
Source 1	Reference/master businesses	source1_id, name, address, country
Source 2	Businesses compared with Source 1	source2_id, name, address, country
Source 3	Additional businesses compared with Source 1	source3_id, name, address, country
Candidate pairs	Plausible comparisons supplied by blocking	source1_id + source2_id/source3_id
Ground truth	Known training matches for supervised learning	source1_id, source2_id, source3_id

4. Preprocessing Expected

Use cleaned/normalized text before calculating similarities: lowercase text, normalize punctuation and whitespace, handle missing values safely, and preserve useful identifying information in addresses. The project guide specifically assigns this preprocessing to Member 1.

5. Similarity Features

Feature	What it captures
Name similarity	Character-level similarity between normalized business names.
Address similarity	Character-level similarity between normalized addresses.
Jaccard similarity	Overlap between token sets.
Edit similarity	Character-level differences such as spelling/edit changes.
TF-IDF + cosine	Text similarity based on informative terms/character patterns.

6. Current Practice Feature Vector

The implemented practice model uses six numerical features:

name_similarity, address_similarity, name_jaccard, address_jaccard, edit_similarity, tfidf_cosine.

The current simple implementation uses Python SequenceMatcher for the name, address and edit similarity signals.

Therefore, those three signals are closely related; a stricter normalized Levenshtein implementation could later be used if the team wants edit similarity to be mathematically distinct.

7. TF-IDF + Cosine

For efficiency, the current practice implementation fits one character-level TF-IDF vectorizer over the combined normalized Source 1 and Source 2 names, with character n-grams of length 2–5. Source vectors are reused while candidate-pair cosine scores are calculated. This is much more efficient than fitting a new vectorizer for every pair.

8. Training Labels

For each candidate pair, the practice pipeline checks the ground-truth relationship. A known true Source 1 ↔ Source 2 pair gets label **1**; other candidate pairs get label **0**. These labels form the supervised-learning target.

9. Machine-Learning Model

The guide recommends starting with a simple classifier such as **Logistic Regression**. The current practice implementation uses Logistic Regression.

Input X: [name_similarity, address_similarity, name_jaccard, address_jaccard, edit_similarity, tfidf_cosine]

Target y: 1 = true match, 0 = non-match candidate.

The trained model produces a match probability with *predict_proba()* and can also produce a binary classification.

10. Training / Testing Flow

1. Build the feature matrix from candidate pairs.
2. Create binary labels from training ground truth.
3. Split the labeled candidates into training and test sets.
4. Train Logistic Regression on the training set.
5. Generate predictions and probabilities on the test set.
6. Inspect precision, recall and other metrics.
7. Save the feature/prediction outputs and, when required, the trained model.

11. Outputs from the Current Practice Pipeline

Output	Meaning
member3_features.csv	Candidate pairs with calculated similarity features and labels.
member3_predictions.csv	Candidate pairs with features, match probability and binary prediction.
member3_model.pkl	Saved Logistic Regression model when the model-saving step is executed.
match_probability	Estimated probability that the candidate pair is a true match.
prediction	Binary decision using the selected threshold; current practice code demonstrates 0.5.

12. Output Row Concept

Conceptually, one prediction row contains:

source1_id | source2_id | similarity features | match_probability | prediction

The exact official submission schema is not defined by this guide and must be taken from the supplied challenge dataset and validator.

13. Evaluation and Threshold

The supplied guide says the challenge uses macro-average F0.5, which gives more importance to precision than recall. The team should avoid forcing weak candidates into matches, and empty matches are allowed when no valid counterpart exists.

$$F0.5 = (1.25 \times \text{Precision} \times \text{Recall}) / (0.25 \times \text{Precision} + \text{Recall})$$

Threshold tuning is Member 4's responsibility. The current practice pipeline uses 0.5 only as a demonstration threshold; the final integrated system should select the threshold using validation data.

14. Current Practice Run

The practice pipeline completed successfully and generated **member3_predictions.csv**. One observed evaluation run had class-1 recall of 0.62 and class-1 F1 of 0.77. Because candidate pairs are heavily dominated by non-matches, overall accuracy should not be treated as the main matching-quality measure.

15. Handoff to Member 4

Member 3 should hand over the candidate-level feature/prediction results and the trained model if the team is using the saved model. Member 4 then connects the pipeline, tunes the threshold, evaluates precision/recall/F0.5, handles final matching logic, creates the required final TSV files and runs the official validator.

16. Key Takeaway

Member 3's core job is:

Candidate pair → Similarity features → Logistic Regression → Match probability/classification.

Member 3 provides the scoring/model layer; final threshold selection and submission integration belong to Member 4 according to the supplied project guide.

Source basis: Business Entity Resolution Hackathon — 4-Member Project Distribution & Step-by-Step Guide supplied in this conversation. Practice-pipeline details reflect the Member 3 implementation developed during this project.